

PROGRAMAÇÃO ORIENTADA A OBJETOS

🕒 Criado em	9 de setembro de 2026 16:58
🏷️ Tags	

1. Antes de POO: o que é um paradigma de programação?

Antes de entender Programação Orientada a Objetos, precisamos entender o que significa **paradigma de programação**.

Um paradigma é uma **forma de pensar, organizar e construir programas**.

Ele estabelece uma maneira de enxergar:

- o problema;
- os dados;
- as operações;
- a organização do código;
- a relação entre as partes do programa.

Uma analogia simples:

Imagine que você precisa organizar uma casa.

Você pode pensar:

┆ "Vou organizar cômodo por cômodo."

Ou:

┆ "Vou separar tudo por tipo de objeto: roupas, livros, utensílios..."

A casa é a mesma. O problema é o mesmo. Mas a **forma de organizar o pensamento** muda.

Na programação acontece algo parecido.

Existem diferentes paradigmas porque diferentes tipos de problemas podem ser pensados e estruturados de maneiras diferentes.

Alguns exemplos:

- Programação Imperativa
- Programação Funcional
- Programação Orientada a Objetos
- Programação Lógica

Um paradigma, portanto, não é simplesmente uma linguagem.

Por exemplo, Java é uma linguagem que dá forte suporte à orientação a objetos, mas também possui recursos de outros estilos de programação.

2. Programação Funcional

Para entender POO, é importante entender pelo menos a ideia da programação funcional.

Na Programação Funcional, as **funções possuem um papel central**.

Uma maneira simplificada de enxergar é:

```
dados
↓
função
↓
resultado
```

A preocupação está muito relacionada à transformação dos dados por meio de funções.

Por exemplo, imagine uma lista:

```
[1, 2, 3, 4, 5]
```

Você poderia ter uma função que transforma cada número:

```
[1, 2, 3, 4, 5]
      ↓
    função
```

↓
[2, 4, 6, 8, 10]

A função recebe dados e produz um resultado.

Um conceito importante associado à programação funcional é a **imutabilidade**: em vez de ficar modificando o estado de uma estrutura constantemente, busca-se trabalhar com novos valores/resultados.

Outro conceito importante é tratar funções como elementos que podem ser utilizados e combinados de maneiras mais flexíveis.

3. Programação Orientada a Objetos

Agora chegamos ao paradigma que é o foco principal da revisão.

A Programação Orientada a Objetos, ou POO, propõe organizar o programa pensando em **objetos que possuem dados e comportamentos**.

Imagine um sistema de biblioteca.

Podemos identificar entidades como:

```
Livro
Usuário
Autor
Empréstimo
Bibliotecário
```

Um **Livro** pode possuir dados:

```
título
ISBN
autor
disponibilidade
```

e comportamentos:

```
emprestar()
devolver()
```

Um **Usuário** pode possuir:

```
nome  
matrícula  
email
```

e comportamentos:

```
realizarEmprestimo()  
devolverLivro()
```

A ideia não é simplesmente transformar substantivos em classes.

O objetivo é **organizar responsabilidades**, fazendo com que cada objeto tenha seus próprios dados e comportamentos relacionados.

4. Funcional x Orientação a Objetos

Agora conseguimos responder à questão 8.

A diferença fundamental está na maneira como cada paradigma organiza o pensamento.

Na programação funcional, existe uma forte centralidade nas:

```
funções + transformação de dados
```

Na orientação a objetos:

```
objetos + estado + comportamento
```

Por exemplo, pense em um sistema de banco.

Na abordagem orientada a objetos podemos ter:

```
Conta  
Cliente  
Transação
```

A classe `Conta` pode ter:

```
private double saldo;

public void depositar(double valor) {
    saldo += valor;
}

public void sacar(double valor) {
    // regras para saque
}
```

O saldo e os comportamentos relacionados à conta ficam organizados dentro da própria classe.

Na programação funcional, poderíamos pensar mais em funções que recebem informações e produzem novos resultados.

É importante não interpretar isso como:

| "Programação funcional é melhor."

ou:

| "POO é melhor."

Não é essa a questão.

São **formas diferentes de estruturar soluções**.

Inclusive, linguagens modernas permitem combinar ideias de diferentes paradigmas.

5. Por que a Orientação a Objetos surgiu?

Agora entra uma questão conceitualmente muito importante:

| Relacione o conceito de Abstração com a motivação para o surgimento do paradigma OO.

Para entender isso, precisamos pensar no problema que a programação tenta resolver.

Quando um sistema fica grande, a realidade que estamos tentando representar também é complexa.

Imagine desenvolver um sistema para uma universidade.

Existem:

```
alunos
professores
disciplinas
turmas
cursos
salas
notas
matrículas
```

Se tentássemos representar absolutamente todos os detalhes dessas coisas, o sistema se tornaria extremamente complexo.

Precisamos selecionar **o que é relevante para o problema**.

É aí que aparece a abstração.

6. O que é abstração?

Abstração é a capacidade de **representar os aspectos relevantes de algo, deixando de lado detalhes que não são necessários naquele contexto**.

Imagine um carro.

Quando você dirige, não precisa conhecer todos os detalhes internos do motor para acelerar.

Você utiliza uma interface:

```
acelerador
freio
volante
câmbio
```

Existe uma enorme quantidade de complexidade por trás disso.

Mas ela está escondida.

Na programação acontece algo parecido.

Imagine:

```
carro.acelerar();
```

Quem utiliza o objeto não necessariamente precisa conhecer todos os detalhes internos de como o carro calcula, controla ou modifica sua velocidade.

A abstração permite trabalhar com uma representação mais simples do problema.

7. Abstração e POO

A orientação a objetos oferece mecanismos que ajudam a representar entidades e conceitos do domínio.

Podemos pegar o sistema universitário:

```
Aluno
Professor
Disciplina
Curso
```

e criar abstrações desses conceitos no software.

Por isso, a abstração está relacionada à motivação da orientação a objetos:

A POO busca facilitar a representação e organização de sistemas complexos por meio de objetos que representam conceitos relevantes do domínio, agrupando dados e comportamentos relacionados.

Isso é muito mais completo do que dizer simplesmente:

"Abstração é esconder detalhes."

Essa frase não está totalmente errada, mas é incompleta.

8. Classe

Agora chegamos a um dos fundamentos.

Uma **classe** é uma definição que descreve características e comportamentos que os objetos daquele tipo poderão possuir.

Por exemplo:

```
public class Carro {  
  
    String modelo;  
    double velocidade;  
  
    public void acelerar() {  
        velocidade += 10;  
    }  
}
```

A classe `Carro` possui:

Atributos

```
String modelo;  
double velocidade;
```

Eles representam características/estado.

Método

```
public void acelerar()
```

Representa um comportamento.

A classe, portanto, funciona como uma **estrutura ou modelo para criação dos objetos**.

9. Objeto

Um objeto é uma **instância de uma classe**.

Podemos criar:

```
Carro c1 = new Carro();  
Carro c2 = new Carro();
```

Agora temos dois objetos.

Eles pertencem à mesma classe:

Carro

mas podem possuir estados diferentes.

Por exemplo:

```
c1
modelo = Civic
velocidade = 80

c2
modelo = Corolla
velocidade = 40
```

10. Estado, comportamento e identidade

Essa é uma forma muito boa de compreender objetos.

Estado

É a situação atual do objeto, determinada pelos valores dos seus atributos.

```
modelo = Civic
velocidade = 80
```

Comportamento

São as ações que o objeto pode realizar.

```
acelerar()
frear()
```

Identidade

É o que permite distinguir uma instância de outra.

Mesmo que:

```
Carro c1 = new Carro();  
Carro c2 = new Carro();
```

tenham exatamente os mesmos valores, continuam sendo objetos diferentes.

11. Classe x objeto

Para guardar:

```
CLASSE  
↓  
define estrutura e comportamento  
  
OBJETO  
↓  
instância concreta dessa classe
```

Uma analogia útil:

```
Classe = projeto/molde  
  
Objeto = elemento criado a partir desse molde
```

Mas lembre-se: essa é uma analogia para facilitar o entendimento. Uma classe não é literalmente apenas um molde; ela também representa uma abstração e define estrutura, comportamento e regras.

12. Encapsulamento

Agora chegamos a outro conceito central.

Encapsulamento significa **organizar e proteger o estado interno de um objeto, controlando como ele pode ser acessado ou modificado**.

Um exemplo ruim seria:

```
public class Conta {  
    public double saldo;  
}
```

Qualquer parte do programa poderia fazer:

```
conta.saldo = -5000;
```

Isso permite que o estado interno seja alterado sem respeitar as regras da conta.

Uma alternativa seria:

```
public class Conta {  
  
    private double saldo;  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        }  
    }  
}
```

Agora o atributo:

```
private double saldo;
```

não fica diretamente disponível para qualquer parte do sistema.

A própria classe controla como seu estado é alterado.

13. Encapsulamento não é simplesmente getter e setter

Isso é importante para uma questão discursiva.

É comum vermos:

```
private String nome;  
  
public String getNome() {  
    return nome;  
}
```

```
}  
  
public void setNome(String nome) {  
    this.nome = nome;  
}
```

Isso é uma forma de implementar acesso controlado.

Mas se você simplesmente cria:

```
get  
set
```

para absolutamente tudo, sem nenhuma regra, você pode acabar expondo demais o estado interno.

O verdadeiro objetivo é:

controlar a interação com o estado interno do objeto e proteger suas regras e invariantes.

14. Encapsulamento, acoplamento e complexidade

Essa é exatamente a parte pedida na questão 4.

Primeiro:

O que é acoplamento?

Acoplamento representa o **grau de dependência entre partes de um sistema**.

Imagine:

```
Classe A  
↓  
conhece muitos detalhes internos da Classe B  
↓  
depende fortemente de B
```

Temos um acoplamento maior.

Agora imagine:

```
Classe A
  ↓
utiliza uma operação pública de B
  ↓
não precisa conhecer seus detalhes internos
```

Existe menor dependência dos detalhes internos.

15. Exemplo prático

Imagine um sistema de pagamento.

Sem encapsulamento, outra classe poderia precisar conhecer detalhes de como o pagamento é processado:

```
verificar saldo
calcular taxa
validar cartão
registrar transação
atualizar status
```

A classe que utiliza o pagamento poderia acabar dependendo de vários detalhes.

Com encapsulamento, podemos ter:

```
pagamento.processar();
```

A classe `Pagamento` fica responsável por coordenar seus próprios detalhes.

Quem utiliza o objeto não precisa saber exatamente como tudo funciona internamente.

Isso reduz o acoplamento porque:

uma classe passa a depender mais da interface/comportamento oferecido por outra e menos de sua implementação interna.

E isso também ajuda a reduzir a complexidade do sistema.

16. Por que isso é importante profissionalmente?

Imagine que a forma de processar o pagamento mude.

Se várias classes conhecem os detalhes internos:

```
Classe A └─  
Classe B └─┬─> detalhes do pagamento  
Classe C └─
```

uma mudança pode exigir alterações em vários lugares.

Se os detalhes estão encapsulados:

```
Classe A ─> processar()  
Classe B ─> processar()  
Classe C ─> processar()  
  
      ↓  
  
Pagamento  
detalhes internos
```

podemos modificar a implementação interna sem necessariamente modificar todas as classes que utilizam aquele comportamento.

Essa é uma das grandes vantagens do encapsulamento.

17. Herança

Agora precisamos entender herança porque ela será essencial para as questões 1, 2 e 6.

Herança permite criar uma classe a partir de outra.

Por exemplo:

```
public class Carro {  
    public void acelerar() {  
        System.out.println("Acelerando");  
    }  
}
```

```
}  
}
```

Podemos criar:

```
public class CarroAlcool extends Carro {  
}
```

Nesse caso:

```
Carro  
↑  
CarroAlcool
```

`CarroAlcool` é uma especialização de `Carro`.

Podemos dizer:

| Um CarroAlcool é um Carro.

Essa relação é conhecida como relação "é um".

18. Superclasse e subclasse

No exemplo:

```
public class Carro {  
}
```

`Carro` é a **superclasse**.

```
public class CarroAlcool extends Carro {  
}
```

`CarroAlcool` é a **subclasse**.

A subclasse pode receber características da superclasse e também adicionar ou modificar comportamentos.

19. Sobrecarga — Overloading

Agora chegamos à questão 1.

Sobrecarga acontece quando temos métodos com o **mesmo nome**, mas com **parâmetros diferentes**.

Por exemplo:

```
public void autonomia() {  
}
```

```
public void autonomia(int distancia) {  
}
```

```
public void autonomia(int distancia, int consumo) {  
}
```

Temos três versões de `autonomia`.

O nome é o mesmo.

Mas os parâmetros são diferentes.

Isso é:

SOBRECARGA / OVERLOADING

20. Como o Java sabe qual método chamar?

Por causa dos argumentos fornecidos.

Se temos:

```
autonomia();
```

é escolhida a versão sem parâmetros.

Se temos:


```
autonomia(10);
```

é escolhida a versão que recebe um `int`.

Se temos:

```
autonomia(10, 20);
```

é escolhida a versão que recebe dois argumentos compatíveis com os parâmetros.

Então:

```
autonomia()  
autonomia(int)  
autonomia(int, int)
```

são métodos sobrecarregados.

21. Atenção: retorno diferente não basta

Isto não caracteriza uma sobrecarga válida apenas por causa do retorno:

```
int calcular() {  
    return 10;  
}
```

e:

```
double calcular() {  
    return 10.0;  
}
```

O Java precisa conseguir diferenciar os métodos pela lista de parâmetros.

Então lembre:

Sobrecarga = mesmo nome + parâmetros diferentes.

22. Sobreposição — Overriding

Agora é outro conceito.

Imagine:

```
public class Carro {  
  
    public void consumo() {  
        System.out.println("Consumo genérico");  
    }  
}
```

A classe filha pode redefinir esse comportamento:

```
public class CarroAlcool extends Carro {  
  
    @Override  
    public void consumo() {  
        System.out.println("Consumo do carro a álcool");  
    }  
}
```

Aqui temos **sobreposição**, também chamada de **overriding**.

A classe filha está fornecendo uma implementação própria de um método herdado.

23. Sobrecarga x sobreposição

Guarde esta comparação:

```
SOBRECARGA  
Overloading  
  
Mesmo nome  
+  
Parâmetros diferentes
```

Enquanto:

SOBREPOSIÇÃO

Overriding

Método herdado

+

subclasse redefine seu comportamento

Exemplo de sobrecarga:

```
autonomia()  
autonomia(10)  
autonomia(10, 20)
```

Exemplo de sobreposição:

```
class Carro {  
    void consumo() {}  
}  
  
class CarroAlcool extends Carro {  
    @Override  
    void consumo() {}  
}
```

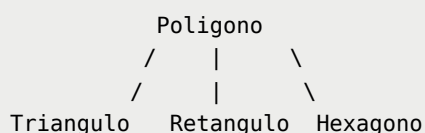
24. Polimorfismo

Agora chegamos a um dos conceitos mais importantes da lista.

"Polimorfismo" significa, literalmente, algo relacionado a **muitas formas**.

Na POO, uma mesma interface/tipo de referência pode ser utilizada para trabalhar com objetos de diferentes classes relacionadas.

Imagine:



Temos uma classe mais geral:

```
Poligono
```

e classes especializadas:

```
Triangulo  
Retangulo  
Hexagono
```

Cada uma possui uma maneira diferente de calcular sua área.

25. O polimorfismo da questão 6

A questão pede:

```
public double calculoArea()
```

Então podemos imaginar:

```
public abstract class Poligono {  
  
    public abstract double calculoArea();  
  
}
```

Depois:

```
public class Triangulo extends Poligono {  
  
    @Override  
    public double calculoArea() {  
        // cálculo da área do triângulo  
    }  
  
}
```

```
public class Retangulo extends Poligono {
```

```
@Override
public double calculoArea() {
    // cálculo da área do retângulo
}
}
```

```
public class Hexagono extends Poligono {

    @Override
    public double calculoArea() {
        // cálculo da área do hexágono
    }
}
```

Agora podemos trabalhar com uma referência do tipo:

```
Poligono
```

e ela pode apontar para diferentes objetos:

```
Poligono p;

p = new Triangulo();
```

ou:

```
p = new Retangulo();
```

ou:

```
p = new Hexagono();
```

Quando fazemos:

```
p.calculoArea();
```

o comportamento correspondente ao objeto real é utilizado.

Isso é polimorfismo.

26. O grande ganho do polimorfismo

Essa é a parte que eu quero que você realmente entenda para a prova.

Imagine que você tenha:

```
Triangulo t;  
Retangulo r;  
Hexagono h;
```

Sem uma estrutura polimórfica, você poderia acabar escrevendo lógica específica para cada tipo:

```
se for Triângulo → calcula triângulo  
se for Retângulo → calcula retângulo  
se for Hexágono → calcula hexágono
```

Com polimorfismo, podemos trabalhar com:

```
Poligono p;
```

e simplesmente pedir:

```
p.calculoArea();
```

Cada objeto sabe como realizar seu próprio cálculo.

Isso favorece:

- reutilização;
- extensibilidade;
- redução de condicionais;
- menor acoplamento;
- código mais flexível;
- facilidade para adicionar novos tipos.

Por exemplo, amanhã podemos criar:

```
public class Pentagono extends Poligono
```

e implementar:

```
calculoArea()
```

O restante do código que trabalha com `Poligono` pode continuar funcionando sem precisar conhecer todos os tipos concretos.

Esse é um dos grandes ganhos do polimorfismo.

27. Tipo da variável e tipo do objeto

Agora precisamos entrar na parte mais traiçoeira da lista.

Veja:

```
Carro c = new CarroAlcool();
```

Existem duas informações diferentes aqui.

A variável:

```
c
```

foi declarada como:

```
Carro
```

Então o **tipo da referência** é `Carro`.

Mas o objeto criado é:

```
new CarroAlcool()
```

Então o **tipo real do objeto** é `CarroAlcool`.

Visualmente:

```
Carro c  
↓
```

tipo da referência

```
new CarroAlcool()  
↓  
tipo real do objeto
```

Isso é extremamente importante para entender polimorfismo.

28. Por que isso é permitido?

Porque:

```
CarroAlcool  
  é um  
  ↓  
  Carro
```

Uma referência de uma superclasse pode apontar para um objeto de uma subclasse compatível.

Por exemplo:

```
Carro c = new CarroAlcool();
```

Isso é permitido.

29. A referência limita o que você pode chamar

Agora imagine:

```
Carro c = new CarroAlcool();
```

Suponha que `Carro` tenha:

```
public void consumo()
```

e `CarroAlcool` tenha:


```
public void consumo()
```

e também:

```
public void característicaExclusiva()
```

Através da referência:

```
c
```

you só pode acessar aquilo que o tipo declarado da referência permite conhecer.

Então:

```
c.consumo();
```

pode ser válido.

Mas:

```
c.característicaExclusiva();
```

pode não ser válido se esse método não existir no tipo `Carro`.

Mesmo que o objeto real seja um `CarroAlcool`.

Essa distinção é fundamental:

O tipo da referência determina o que pode ser acessado em tempo de compilação, enquanto o tipo real do objeto influencia qual implementação sobrescrita será executada.

Essa frase é excelente para entender a questão 2.

30. Classe abstrata

Agora chegamos à questão 3.

Uma classe abstrata é uma classe que representa uma abstração que **não deve ser instanciada diretamente**.

Por exemplo:

```
public abstract class Poligono {  
}
```

Não podemos fazer:

```
Poligono p = new Poligono();
```

Isso não é permitido porque `Poligono` é abstrata.

Mas podemos declarar uma referência:

```
Poligono p;
```

E podemos fazer:

```
Poligono p = new Triangulo();
```

desde que `Triangulo` seja uma classe concreta compatível.

31. Por que uma variável pode ser do tipo abstrato?

Porque uma variável/referência e um objeto são coisas diferentes.

Quando fazemos:

```
Poligono p;
```

não estamos criando um `Poligono`.

Estamos apenas declarando uma referência capaz de apontar para um objeto compatível.

Depois:

```
p = new Triangulo();
```

Agora a referência `p` aponta para um objeto concreto:

```
p
↓
Triangulo
```

Então a frase da questão significa:

Podemos utilizar uma classe abstrata como tipo de referência porque ela representa uma abstração comum para suas subclasses. Entretanto, não podemos criar diretamente uma instância dessa classe porque ela representa uma classe incompleta/abstrata que deve ser especializada por classes concretas.

32. Métodos abstratos

Uma classe abstrata pode também possuir métodos abstratos.

Por exemplo:

```
public abstract class Poligono {

    public abstract double calculoArea();

}
```

Aqui estamos dizendo:

Todo polígono deve saber calcular sua área, mas a forma de realizar esse cálculo dependerá do tipo específico de polígono.

Então:

Triangulo

implementa de uma maneira.

Retangulo

de outra.

Hexagono

de outra.

Isso combina perfeitamente:

```
ABSTRAÇÃO
↓
Poligono
↓
define o comportamento esperado
↓
POLIMORFISMO
↓
cada subclasse implementa sua própria forma
```

33. Agora conseguimos entender a questão 2

A questão 2 mistura praticamente tudo que estudamos.

Ela provavelmente quer testar se você sabe identificar:

- herança;
- compatibilidade entre tipos;
- instanciação;
- tipo da referência;
- tipo real do objeto;
- existência de métodos;
- sobrecarga;
- polimorfismo.

Por isso eu não quero que você memorize uma tabela de V/F.

O importante é aprender a analisar uma linha.

Por exemplo:

```
Carro c = new CarroAlcool();
```

Pergunte:

1. **CarroAlcool** é compatível com **Carro** ?

Se:

```
CarroAlcool extends Carro
```

sim.

Então a atribuição é possível.

Agora:

```
CarroFlex c1 = new CarroGasolina();
```

Você precisa perguntar:

| **CarroGasolina** é um **CarroFlex** ?

Se o diagrama não indicar essa relação, a atribuição não é válida.

E é justamente por isso que **precisamos da Figura 1 para responder definitivamente a tabela.**

34. Como analisar chamadas de métodos

Quando encontrar:

```
c.autonomia();
```

não saia olhando apenas para o objeto.

Pergunte:

Primeiro:

Qual é o tipo da referência?

Carro c

Segundo:

Existe um método compatível com essa chamada no tipo Carro?

Terceiro:

Existe sobreposição?

Se existir, qual implementação será executada pelo objeto real?

Quarto:

Os parâmetros estão corretos?

Por exemplo:

```
autonomia()
```

é diferente de:

```
autonomia(10)
```

que é diferente de:

```
autonomia(10, 20)
```

Isso conecta diretamente sobrecarga e polimorfismo.

35. Um mapa para não confundir os conceitos

Quando você olhar para uma questão, tente identificar onde ela está:

```
CLASSE
↓
Define estrutura e comportamento

OBJETO
↓
Instância da classe
```

HERANÇA

↓

Uma classe é especializada a partir de outra

ENCAPSULAMENTO

↓

Controla e protege o estado interno

ABSTRAÇÃO

↓

Representa o que é relevante e esconde complexidade desnecessária

SOBRECARGA

↓

Mesmo nome + parâmetros diferentes

SOBREPOSIÇÃO

↓

Subclasse redefine comportamento herdado

POLIMORFISMO

↓

Uma referência/tipo comum pode trabalhar com diferentes objetos relacionados

CLASSE ABSTRATA

↓

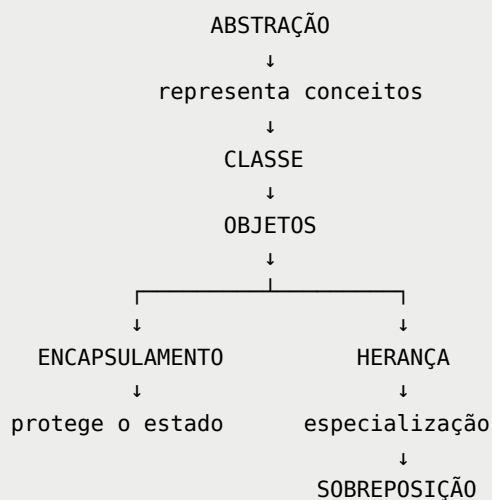
Pode servir como tipo e ser herdada

Mas não pode ser instanciada diretamente

36. Como esses conceitos se relacionam

O mais importante é não estudar tudo como caixinhas independentes.

Imagine:



↓
POLIMORFISMO

A sobrecarga entra como outra possibilidade:

Métodos
↓
mesmo nome
↓
parâmetros diferentes
↓
SOBRECARGA

E a classe abstrata pode servir como uma abstração comum:

```
      Poligono
    (abstrata)
   /   |   \
  /    |    \
Triangulo Retangulo Hexagono
  \    |    /
   \   |   /
      POLIMORFISMO
```

37. Um exemplo integrando praticamente tudo

Vamos imaginar um sistema de veículos.

Temos:

```
public abstract class Carro {

    private String modelo;

    public Carro(String modelo) {
        this.modelo = modelo;
    }

    public String getModelo() {
        return modelo;
    }
}
```



```
}

    public abstract double consumo();
}
```

Aqui temos:

Abstração

`Carro` representa uma ideia geral de carro.

Classe abstrata

```
abstract class Carro
```

Não podemos fazer:

```
new Carro(...)
```

Encapsulamento

```
private String modelo;
```

O estado interno é protegido.

Método abstrato

```
public abstract double consumo();
```

Toda classe concreta deverá fornecer uma implementação.

Agora:

```
public class CarroAlcool extends Carro {

    public CarroAlcool(String modelo) {
        super(modelo);
    }

    @Override
```

```
    public double consumo() {  
        return 8.5;  
    }  
}
```

Temos:

Herança

```
extends Carro
```

Sobreposição

```
@Override  
public double consumo()
```

Polimorfismo

Podemos fazer:

```
Carro c = new CarroAlcool("Modelo X");
```

A referência é:

```
Carro
```

O objeto é:

```
CarroAlcool
```

E:

```
c.consumo();
```

utiliza a implementação de `CarroAlcool`.

38. E onde entra a sobrecarga?

Podemos ter:

```
public double autonomia() {  
    return 100;  
}
```

e:

```
public double autonomia(double distancia) {  
    return distancia / consumo();  
}
```

Agora temos:

```
autonomia()  
autonomia(double)
```

Isso é **sobrecarga**.

Perceba que sobrecarga e sobreposição podem aparecer no mesmo sistema, mas são mecanismos diferentes.

39. Uma forma de pensar na hora da prova

Se aparecer uma questão conceitual, não tente imediatamente lembrar uma definição decorada.

Faça perguntas.

Se aparecer "classe":

| É a definição de quê?

Se aparecer "objeto":

| É uma instância de quê?

Se aparecer "encapsulamento":

O que está sendo protegido? Quem pode acessar? Por que isso reduz dependência?

Se aparecer "abstração":

O que estou representando e quais detalhes não preciso expor?

Se aparecer "herança":

Qual classe é mais geral e qual é a especialização?

Se aparecer "sobrecarga":

O nome é igual e os parâmetros mudaram?

Se aparecer "sobreposição":

Uma subclasse está redefinindo um comportamento herdado?

Se aparecer "polimorfismo":

Um tipo comum está sendo usado para trabalhar com diferentes objetos?

Se aparecer "classe abstrata":

Posso declarar uma referência? Sim. Posso fazer `new` diretamente? Não.

40. Como responder questões discursivas

Como ainda existe a possibilidade de a prova ser aberta, você precisa conseguir **desenvolver uma resposta**, não apenas escrever uma definição.

Uma estrutura excelente é:

1. Defina o conceito.
2. Explique como ele funciona.
3. Dê um exemplo.
4. Explique a vantagem/objetivo.
5. Relacione com outro conceito quando fizer sentido.

Por exemplo, se perguntarem:

| Explique encapsulamento.

Uma resposta superficial seria:

| Encapsulamento é esconder os atributos usando `private`.

Uma resposta muito melhor seria:

| Encapsulamento é o princípio de controlar o acesso ao estado interno de um objeto e concentrar nele as regras relacionadas aos seus dados. Em Java, isso pode ser implementado utilizando modificadores de acesso como `private` e métodos públicos para controlar as operações permitidas. Isso evita que outras partes do sistema dependam diretamente dos detalhes internos da classe, reduzindo o acoplamento e ajudando a controlar a complexidade. Por exemplo, uma classe `Conta` pode manter o atributo `saldo` privado e permitir alterações somente por meio de métodos como `depositar()` e `sacar()`, que podem validar as regras da conta.

Percebe a diferença?

A segunda resposta demonstra **entendimento**.

41. O que eu considero mais importante dessa lista

Se você tiver pouco tempo para revisar, eu colocaria a prioridade assim:

Muito importante

1. Polimorfismo

Entender:

```
Carro c = new CarroAlcool();
```

e:

```
c.consumo();
```

2. Sobrecarga x sobreposição

Você precisa conseguir diferenciar imediatamente:

```
Overloading ≠ Overriding
```

3. Classe abstrata

Principalmente:

```
Poligono p;
```

versus:

```
new Poligono();
```

4. Encapsulamento + acoplamento + complexidade

Porque o professor explicitamente pediu essa relação.

5. Abstração

Principalmente sua relação com a motivação da POO.

Depois

6. Classe x objeto

7. Herança

8. Funcional x POO

Esses também são importantes, mas funcionam como base para os demais.

42. O grande mapa mental para a prova

Se você conseguir explicar este mapa com suas próprias palavras, você estará muito mais preparada:

```
PARADIGMA
↓
forma de pensar/organizar a programação
↓
```

```
P00
↓
modela conceitos por meio de objetos
↓
CLASSES
↓
definem atributos e comportamentos
↓
OBJETOS
↓
instâncias das classes
↓
ABSTRAÇÃO
↓
representa apenas o que é relevante
↓
ENCAPSULAMENTO
↓
protege/controla o estado interno
↓
HERANÇA
↓
permite especialização
↓
SOBREPOSIÇÃO
↓
subclasse redefine comportamento
↓
POLIMORFISMO
↓
mesma operação pode produzir comportamentos
diferentes conforme o objeto
```

E paralelamente:

```
SOBRECARGA
↓
mesmo método
+
parâmetros diferentes
```

43. E uma última distinção que eu quero que você leve para a prova

Não confunda:

```
ABSTRAÇÃO
```

com:

ENCAPSULAMENTO

Eles se relacionam, mas não são a mesma coisa.

Abstração responde principalmente:

| "O que eu preciso representar e quais detalhes são relevantes?"

Encapsulamento responde principalmente:

| "Como eu organizo e controlo o acesso aos detalhes internos dessa representação?"

Exemplo:

Carro

Abstração

Decidimos que para o sistema precisamos representar:

```
modelo  
velocidade  
acelerar()  
frear()
```

Não precisamos representar todos os detalhes físicos de um carro.

Encapsulamento

Decidimos que:

```
private double velocidade;
```

e que a velocidade será modificada por operações controladas:

```
acelerar();  
frear();
```


Então:

abstração = o que representar

encapsulamento = como proteger/controlar a representação

Essa diferença é muito boa para questões discursivas.
